

Domain-driven design

Domain-driven design (DDD) is a software design approach^[1] that focuses on modeling software to match a domain according to input from that domain's experts.^[2] DDD is against the idea of having a single unified model; instead it divides a large system into bounded contexts, each of which have their own model.^{[3][4]}

Under domain-driven design, the structure and language of software code (class names, class methods, class variables) should match the business domain. For example: if software processes loan applications, it might have classes like "loan application", "customers", and methods such as "accept offer" and "withdraw".

Domain-driven design is predicated on the following goals:

- placing the project's primary focus on the core domain and domain logic layer;
- basing complex designs on a model of the domain;
- initiating a creative collaboration between technical and domain experts to iteratively refine a conceptual model that addresses particular domain problems.

Critics of domain-driven design argue that developers must typically implement a great deal of isolation and encapsulation to maintain the model as a pure and helpful construct. While domain-driven design provides benefits such as maintainability, Microsoft recommends it only for complex domains where the model provides clear benefits in formulating a common understanding of the domain.^[5]

The term was coined by Eric Evans in his book of the same name published in 2003.^[3]

Overview

Domain-driven design articulates a number of high-level concepts and practices.^[3]

Of primary importance is a *domain* of the software, the subject area to which the user applies a program. Software's developers build a *domain model*: a system of abstractions that describes selected aspects of a domain and can be used to solve problems related to that domain.

These aspects of domain-driven design aim to foster a *common language* shared by domain experts, users, and developers—the *ubiquitous language*. The ubiquitous language is used in the domain model and for describing system requirements.

Ubiquitous language is one of the pillars of DDD together with *strategic design* and *tactical design*.

In domain-driven design, the domain layer is one of the common layers in an object-oriented multilayered architecture.

Kinds of models

Domain-driven design recognizes multiple kinds of models. For example, an *entity* is an object defined not by its attributes, but its identity. As an example, most airlines assign a unique number to seats on every flight: this is the seat's identity. In contrast, a *value object* is an immutable object that contains attributes but has no conceptual identity. When people exchange business cards, for instance, they only care about the information on the card (its attributes) rather than trying to distinguish between each unique card.

Models can also define *events* (something that happened in the past). A domain event is an event that domain experts care about. Models can be bound together by a root entity to become an *aggregate*. Objects outside the aggregate are allowed to hold references to the root but not to any other object of the aggregate. The aggregate root checks the consistency of changes in the aggregate. Drivers do not have to individually control each wheel of a car, for instance: they simply drive the car. In this context, a car is an aggregate of several other objects (the engine, the brakes, the headlights, etc.).

Working with models

In domain-driven design, an object's creation is often separated from the object itself.

A *repository*, for instance, is an object with methods for retrieving domain objects from a data store (e.g. a database). Similarly, a *factory* is an object with methods for directly creating domain objects.

When part of a program's functionality does not conceptually belong to any object, it is typically expressed as a service.

Event types

There are different types of events in DDD, and opinions on their classification may vary. According to Yan Cui, there are two key categories of events:^[6]

Domain Events

Domain events signify important occurrences within a specific business domain. These events are restricted to a bounded context and are vital for preserving business logic. Typically, domain events have lighter payloads, containing only the necessary information for processing. This is because event listeners are generally within the same service, where their requirements are more clearly understood.^[6]

Integration Events

On the other hand, integration events serve to communicate changes across different bounded contexts. They are crucial for ensuring data consistency throughout the entire system. Integration events tend to have more complex payloads with additional attributes, as the needs of potential listeners can differ significantly. This often leads to a more thorough approach to communication, resulting in overcommunication to ensure that all relevant information is effectively shared.^[6]

Context Mapping patterns

Context Mapping identifies and defines the boundaries of different domains or subdomains within a larger system. It helps visualize how these contexts interact and relate to each other. Below are some patterns, according to Eric Evans:^[7]

- Partnership: *"forge a partnership between the teams in charge of the two contexts. Institute a process for coordinated planning of development and joint management of integration", when "teams in two contexts will succeed or fail together"*
- Shared Kernel: *"Designate with an explicit boundary some subset of the domain model that the teams agree to share. Keep this kernel small."*
- Customer/Supplier Development: *"Establish a clear customer/supplier relationship between the two teams", when "two teams are in [a] upstream-downstream relationship"*
- Conformist: *"Eliminate the complexity of translation [...] choosing conformity enormously simplifies integration", when a custom interface for a downstream subsystem isn't likely to happen*
- Anticorruption Layer: *"create an isolating layer to provide your system with functionality of the upstream system in terms of your own domain model"*
- Open-host Service: *"a protocol that gives access to your subsystem as a set of services", in case it's necessary to integrate one subsystem with many others, making custom translations between subsystems infeasible*
- Published Language: *"a well--documented shared language that can express the necessary domain information as a common medium of communication", e.g. data interchange standards in various industries*
- Separate Ways: *"a bounded context [with] no connection to the others at all, allowing developers to find simple, specialized solutions within this small scope"*
- Big Ball of Mud:^[8] *"a boundary around the entire mess" when there are no real boundaries to be found when surveying an existing system*

Relationship to other ideas

Although domain-driven design is not inherently tied to object-oriented approaches, in practice, it exploits the advantages of such techniques. These include entities/aggregate roots as receivers of commands/method invocations, the encapsulation of state within foremost aggregate roots, and on a higher architectural level, bounded contexts.

As a result, domain-driven design is often associated with Plain Old Java Objects and Plain Old CLR Objects, which are technical implementation details, specific to Java and the .NET Framework respectively. These terms reflect a growing view that domain objects should be defined purely by the business behavior of the domain, rather than by a more specific technology framework.

Similarly, the naked objects pattern holds that the user interface can simply be a reflection of a good enough domain model. Requiring the user interface to be a direct reflection of the domain model will force the design of a better domain model.^[9]

Domain-driven design has influenced other approaches to software development.

Domain-specific modeling, for instance, is domain-driven design applied with domain-specific languages. Domain-driven design does not specifically require the use of a domain-specific language, though it could be used to help define a domain-specific language and support domain-specific multimodeling.

In turn, aspect-oriented programming makes it easy to factor out technical concerns (such as security, transaction management, logging) from a domain model, letting them focus purely on the business logic.

Model-driven engineering and architecture

While domain-driven design is compatible with model-driven engineering and model-driven architecture,^[10] the intent behind the two concepts is different. Model-driven architecture is more concerned with translating a model into code for different technology platforms than defining better domain models.

However, the techniques provided by model-driven engineering (to model domains, to create domain-specific languages to facilitate the communication between domain experts and developers,...) facilitate domain-driven design in practice and help practitioners get more out of their models. Thanks to model-driven engineering's model transformation and code generation techniques, the domain model can be used to generate the actual software system that will manage it.^[11]

Command Query Responsibility Segregation

Command Query Responsibility Segregation (CQRS) is an architectural pattern for separating reading data (a 'query') from writing to data (a 'command'). CQRS derives from Command and Query Separation (CQS), coined by Bertrand Meyer.

Commands mutate state and are approximately equivalent to method invocation on aggregate roots or entities. Queries read state but do not mutate it.

While CQRS does not require domain-driven design, it makes the distinction between commands and queries explicit with the concept of an aggregate root. The idea is that a given aggregate root has a method that corresponds to a command and a command handler invokes the method on the aggregate root.

The aggregate root is responsible for performing the logic of the operation and either yielding a failure response or just mutating its own state that can be written to a data store. The command handler pulls in infrastructure concerns related to saving the aggregate root's state and creating needed contexts (e.g., transactions).

Event storming

Event storming is a collaborative, workshop-based modeling technique which can be used as a precursor in the context of Domain-Driven Design (DDD) to identify and understand domain events. This interactive discovery process involves stakeholders, domain experts, and developers working together to visualize the flow of domain events, their causes, and their effects, fostering a shared understanding of the domain. The technique often uses color-coded sticky notes to represent different elements, such as domain events, aggregates, and external systems, facilitating a clear and structured exploration of the

domain. Event storming can aid in discovering subdomains, bounded contexts, and aggregate boundaries, which are key constructs in DDD. By focusing on 'what happens' in the domain, the technique can help uncover business processes, dependencies, and interactions, providing a foundation for implementing DDD principles and aligning system design with business goals.^{[12][13]}

Event sourcing

Event sourcing is an architectural pattern in which entities track their internal state not by means of direct serialization or object-relational mapping, but by reading and committing events to an event store.

When event sourcing is combined with CQRS and domain-driven design, aggregate roots are responsible for validating and applying commands (often by having their instance methods invoked from a Command Handler), and then publishing events. This is also the foundation upon which the aggregate roots base their logic for dealing with method invocations. Hence, the input is a command and the output is one or many events which are saved to an event store, and then often published on a message broker for those interested (such as an application's view).

Modeling aggregate roots to output events can isolate internal state even further than when projecting read-data from entities, as in standard *n*-tier data-passing architectures. One significant benefit is that axiomatic theorem provers (e.g. Microsoft Contracts and CHESS^[14]) are easier to apply, as the aggregate root comprehensively hides its internal state. Events are often persisted based on the version of the aggregate root instance, which yields a domain model that synchronizes in distributed systems through optimistic concurrency.

Mapping Bounded Contexts to Microservices

A bounded context, a fundamental concept in Domain-Driven Design (DDD), defines a specific area within which a domain model is consistent and valid, ensuring clarity and separation of concerns.^[15] In microservices architecture, a bounded context often maps to a microservice, but this relationship can vary depending on the design approach. A one-to-one relationship, where each bounded context is implemented as a single microservice, is typically ideal as it maintains clear boundaries, reduces coupling, and enables independent deployment and scaling. However, other mappings may also be appropriate: a one-to-many relationship can arise when a bounded context is divided into multiple microservices to address varying scalability or other operational needs, while a many-to-one relationship may consolidate multiple bounded contexts into a single microservice for simplicity or to minimize operational overhead. The choice of relationship should balance the principles of DDD with the system's business goals, technical constraints, and operational requirements.^[16]

Notable tools

Although domain-driven design does not depend on any particular tool or framework, notable examples include:

- Actifsource, a plug-in for Eclipse which enables software development combining DDD with model-driven engineering and code generation.

- Context Mapper, a Domain-specific language and tools for strategic and tactic DDD.^[17]
- CubicWeb, an open source semantic web framework entirely driven by a data model. High-level directives allow to refine the data model iteratively, release after release. Defining the data model is enough to get a functioning web application. Further work is required to define how the data is displayed when the default views are not sufficient.
- OpenMDX, an open-source, Java-based, MDA Framework supporting Java SE, Java EE, and .NET. OpenMDX differs from typical MDA frameworks in that *"use models to directly drive the runtime behavior of operational systems"*.
- Restful Objects, a standard for mapping a Restful API onto a domain object model (where the domain objects may represent entities, view models, or services). Two open source frameworks (one for Java, one for .NET) can create a Restful Objects API from a domain model automatically, using reflection.

See also

- Data mesh, a domain-oriented data architecture
- Event storming
- Knowledge representation
- Ontology (information science)
- Semantic analysis (knowledge representation)
- Semantic networks
- Semantics
- C4 model
- Strongly typed identifier
- Integrated design
- Systems science

References

1. Millet, Scott; Tune, Nick (2015). *Patterns, Principles, and Practices of Domain-Driven Design*. Indianapolis: Wrox. ISBN 978-1-118-71470-6.
2. Vernon, Vaughn (2013). *Implementing Domain-Driven Design*. Upper Sadle River, NJ: Addison-Wesley. p. 3. ISBN 978-0-321-83457-7.
3. Evans, Eric (August 22, 2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software* (http://dddcommunity.org/book/evans_2003/). Boston: Addison-Wesley. ISBN 978-032-112521-7. Retrieved 2012-08-12.
4. martinekuan. "Using tactical DDD to design microservices - Azure Architecture Center" (<https://learn.microsoft.com/en-us/azure/architecture/microservices/model/tactical-ddd>). *learn.microsoft.com*. Retrieved 2024-09-07.
5. Microsoft Application Architecture Guide, 2nd Edition. Retrieved from <http://msdn.microsoft.com/en-us/library/ee658117.aspx#DomainModelStyle>.
6. Cui, Yan. *Serverless Architectures on AWS*. Manning. ISBN 978-1617295423.
7. Evans, Eric. *Domain-Driven Design Reference: Definitions and Pattern Summaries*. ISBN 978-1457501197.
8. Foote, Brian; Yoder, Joseph (1999), *Big Ball of Mud* (<http://www.laputan.org/mud/mud.html>), retrieved 2025-05-09
9. Haywood, Dan (2009), *Domain-Driven Design using Naked Objects* (<http://www.pragprog.com/titles/dhnako/domain-driven-design-using-naked-objects>), Pragmatic Programmers.

10. MDE can be regarded as a superset of MDA (<https://modeling-languages.com/clarifying-concepts-mbe-vs-mde-vs-mdd-vs-mda/>)
11. Cabot, Jordi (2017-09-11). "Comparing Domain-Driven Design with Model-Driven Engineering" (<https://modeling-languages.com/comparing-domain-driven-design-model-driven-engineering/>). *Modeling Languages*. Retrieved 2021-08-05.
12. *Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy*. ISBN 978-1098100131.
13. *Open Agile Architecture™ - A Standard of The Open Group*. ISBN 9789401807265.
14. a MS bug finding tool (<https://www.microsoft.com/en-us/research/project/chess-find-and-reproduce-heisenbugs-in-concurrent-programs/>)
15. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. 2020. ISBN 978-1492043454.
16. *Building Microservices by Sam Newman*. ISBN 978-1492034025.
17. Stefan Kapferer and Olaf Zimmermann: Domain-driven Service Design - Context Modeling, Model Refactoring and Contract Generation, 14th Symposium and Summer School On Service-Oriented Computing (SommerSoC 2020)[1]

External links

- *Domain Driven Design, Definitions and Pattern Summaries* (https://domainlanguage.com/wp-content/uploads/2016/05/DDD_Reference_2015-03.pdf) (PDF), Eric Evans, 2015
- DDD Crew on GitHub (<https://github.com/ddd-crew>): Bounded Context Canvas, Aggregate Canvas, Modeling Process and more repositories
- *An Introduction to Domain Driven Design* (<http://www.methodsandtools.com/archive/archive.php?id=97>), Methods & tools
- Implementing Aggregate root in C# language (<https://www.codeproject.com/Articles/1055514/Aggregate-Root-Pattern-in-Csharp>)
- Context Mapper: A Modeling Framework for Strategic Domain-driven Design (Tool, Tutorials, DDD Modeling Examples) (<https://contextmapper.org/>)
- Strategic DDC Activity in Design Practice Repository (DPR) (<https://socadk.github.io/design-practice-repository/activities/DPR-StrategicDDD.html>) and Tactic DDC Activity in Design Practice Repository (DPR) (<https://socadk.github.io/design-practice-repository/activities/DP-R-TacticDDD.html>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Domain-driven_design&oldid=1356273375"